Byte Me: System Evaluation and Measures of Success

Development Team

February 17, 2026

Abstract

This document outlines the five key measures of success used to evaluate the "Byte Me" food rescue platform. Each measure is defined with specific criteria and supported by evidence from the system's codebase, ensuring reliability, accuracy, and environmental impact.

1 Marketplace Reliability

1.1 Oversell Rate (Target: 0%)

Definition: The system must never allow more reservations than the available quantity of a bundle.

Evidence: The `BundlePosting` entity enforces a strict logic check before any reservation is permitted. The `canReserve` method explicitly verifies inventory levels.

```java
// Check if can reserve
public boolean canReserve(int qty) {
    return status == Status.ACTIVE && (quantityReserved + qty) <= quantityTotal;
}
```

Listing 1: BundlePosting.java Inventory Logic

Furthermore, the `OrderController` transactionally locks this process. If `canReserve(1)` returns false, the system immediately rejects the request with a `400 Bad Request`, ensuring the oversell rate remains strictly at 0.

1.2 Handling Expiries and No-Shows

Definition: The lifecycle of every bundle must be accounted for, distinguishing between successful collections, expired goods, and user no-shows.

Evidence:

- **Status Tracking:** The `Reservation.Status` enum includes specific states for `NO_SHOW`, `EXPIRED`, and `CANCELLED`.
- **Automated Cleanup:** The `BundlePostingRepository` includes a query to identify expired bundles automatically:

```java
@Query("SELECT b FROM BundlePosting b WHERE b.status = 'ACTIVE' AND b.
    pickupEndAt < :now")
List<BundlePosting> findExpired(Instant now);
```

Listing 2: Finding Expired Bundles

This ensures that inventory lists remain fresh and accurate without manual intervention.

2 Forecast Accuracy

2.1 Methodology

We compare our chosen predictive model against standard baselines using a Held-Out Evaluation approach (80% training data, 20% evaluation data).

2.2 Metrics & Baselines

The `ForecastService` calculates three key error metrics for every model run:

1. **MAE (Mean Absolute Error):** Average magnitude of reservation errors.
2. **RMSE (Root Mean Squared Error):** Penalizes larger outliers.
3. **Brier Score:** Evaluates the accuracy of probabilistic no-show predictions.

These metrics are calculated by comparing the **Chosen Model (EMA-Weighted)** against two baselines:

- **Moving Average (4-Week):** A simple sliding window average.
- **Seasonal Naive:** Predicts based on the same ISO Day of Week from history.

```
// Compare models and save metrics
Map<String, Object> maMetrics = Map.of(
    "MAE_reservations", calculateMAE(maPredicted, actualRes),
    "RMSE_reservations", calculateRMSE(maPredicted, actualRes),
    "Brier_no_show", calculateBrierScore(maNoShowPred, actualNoShow));
```

Listing 3: Model Comparison Logic

The system stores these results in the `ForecastRun` entity to allow for continuous performance monitoring.

3 Waste Impact & Carbon Reduction

3.1 Estimated Waste Avoided

Measure: The total weight of food diverted from landfills.

Evidence: The `SellerMetricsWeekly` entity tracks `wasteAvoidedGrams`. This is calculated by multiplying the number of `COLLECTED` reservations by the `estimatedWeightGrams` defined in the bundle posting.

3.2 Real-World Example: CO2e Prevention

To illustrate, consider the sample data for "Sourdough & Co Bakery" used in our forecast demo.

- **Scenario:** A bakery posts surplus bundles with an estimated weight of 800g each.
- **Observations:** On Jan 30, 2026, the bakery observed 16 reservations with a 9% no-show rate.
- **Food Waste Avoided:**

$$\text{Mass} = 16 \times (1 - 0.09) \times 0.8 \text{ kg} \approx 11.65 \text{ kg}$$

- **Carbon Impact:** Using a standard conversion factor of 2.5 kg CO2e per kg of food waste prevented (WRAP, 2023):

$$\text{CO2e Saved} = 11.65 \text{ kg} \times 2.5 = \mathbf{29.1 \text{ kg CO2e}}$$

This metric is aggregated weekly and displayed on the seller dashboard, providing tangible feedback on their environmental contribution. The `CO2_SAVER` badge is explicitly awarded when organizations reach 10 collections, further reinforcing this goal.

4 User Experience (Task-Based)

4.1 Reserving and Claiming

Target: Sub-second response times for reservation and collection tasks.

Evidence:

- **Instant Reservation:** The `create` order endpoint generates a secure 6-digit `claimCode` immediately upon success.
- **Secure Collection:** The collection process uses a hashed verification system. When an organisation presents a code, the backend verifies it using `passwordEncoder.matches()`.

```
if (!passwordEncoder.matches(claimReq.getClaimCode(), reservation.
    getClaimCodeHash())) {
    return ResponseEntity.badRequest().body("Invalid claim code");
}
```

Listing 4: Claim Code Verification

This approach balances high security with the speed required for retail environments.

5 Gamification & Engagement

5.1 Measures

- **Retention:** Tracking `currentStreakWeeks` via the `OrganisationStreakCache`.
- **Milestones:** Awarding badges like `STREAK_4` and `CO2_SAVER`.

Evidence: The system automatically updates streak counters and checks for badge eligibility after every successful collection, providing immediate positive reinforcement to the user.